

Reinforcement Learning for Resource-Aware Lane Defense in Plants vs. Zombies

Group 7

Muhammad Aufa Helfiandri

Siyu Lu

Yifan Deng

Abstract

This project investigates reinforcement learning for resource-constrained sequential decision-making using a simplified *Plants vs. Zombies*-inspired lane-defense environment. The task requires an agent to manage limited resources, place defensive units on a grid, respond to stochastic enemy waves, and optimize long-term survival. We implement and compare DDQN, Actor-Critic, heuristic, and baseline agents using survival-based metrics such as ticks survived and waves reached. The results show that the Actor-Critic agent achieves the strongest overall performance and learns meaningful strategies such as early resource investment, delayed defense, and emergency stalling. We also find that richer state representations, interim rewards, and appropriate training difficulty improve learning. These results suggest that reinforcement learning can be effective for similar resource-allocation problems, but performance depends strongly on environment design, reward shaping, and action-space structure.

I. INTRODUCTION

Reinforcement learning (RL) is a machine learning approach for solving sequential decision-making problems. In such problems, an agent repeatedly observes the current state of an environment, selects an action, receives feedback through rewards or penalties, and updates its behavior in order to maximize long-term return [?]. This framework is especially useful when decisions have delayed consequences, when the environment changes over time, and when the best action cannot be determined only from immediate outcomes.

Many practical problems have this structure. Examples include resource allocation, scheduling, robotics, traffic control, inventory management, and strategic game playing. In these domains, an agent must often balance short-term needs with long-term objectives. For instance, spending a limited resource immediately may solve a current problem but reduce future flexibility, while saving resources may improve future performance but create short-term risk. Reinforcement learning is therefore suitable for studying problems that involve planning, uncertainty, resource constraints, and adaptive decision-making.

This project uses a simplified *Plants vs. Zombies*-inspired environment as a case study for this class of reinforcement learning problems. *Plants vs. Zombies* is a tower-defense game in which the player protects a house from waves of zombies by placing different plants on a grid [?]. Although the game setting is fictional, the underlying decision problem contains several general RL challenges. The agent must allocate limited resources, choose actions under time pressure, respond to stochastic enemy behavior, and make decisions whose effects may only become visible later in the episode.

In our environment, the agent controls the plant side of the game. The game is represented as a 5×9 grid with five lanes, where zombies advance toward the house and the agent places plants to defend the lawn. The environment includes sun-based resource management, plant cooldowns, wave-based zombie spawning, multiple plant types, and different zombie behaviors. The agent chooses from a discrete action space of 316 possible actions, consisting of one wait action and all possible plant placements across seven plant types and the 5×9 grid. This makes the environment a compact but challenging testbed for studying how reinforcement learning agents handle spatial placement, resource management, and delayed rewards.

The main research question of this project is:

Research Questions

Main question: To what extent can reinforcement learning agents learn effective resource-management and spatial decision-making strategies in a stochastic lane-defense environment?

Case study: We investigate this question using a simplified *Plants vs. Zombies*-inspired environment.

Sub-questions:

- How do reinforcement learning agents compare with simpler non-learning and rule-based agents?
- How does a richer state representation affect agent performance?
- How do DDQN and Actor-Critic compare in this environment?
- How does reward design influence the strategies learned by the agent?

To answer these questions, we formulate the environment as a reinforcement learning problem by defining its state space, action space, reward function, and episode structure. The state representation is expanded from a simpler 58-dimensional version

to a 126-dimensional feature vector that includes plant positions, zombie health, sun amount, plant availability, cooldowns, mower status, nearest zombie positions, lane plant health, and game progress. This richer representation is intended to provide the agent with more information about both immediate threats and long-term game progression.

We implement and compare two reinforcement learning agents: Double Deep Q-Network (DDQN) and Actor-Critic. DDQN represents a value-based approach that estimates the long-term value of each action, while Actor-Critic combines policy learning with value estimation. In addition, we compare these learning agents with simpler non-learning baselines, such as random action selection and doing nothing. If included in the final experiments, a rule-based heuristic agent can also serve as a stronger comparison point for evaluating whether RL agents learn strategies beyond simple manually designed rules.

The contribution of this project is threefold. First, we design and implement a stochastic lane-defense environment that captures several general RL challenges, including resource management, spatial decision-making, delayed rewards, and increasing difficulty. Second, we study how different reinforcement learning methods behave in this environment by comparing DDQN and Actor-Critic under the same task formulation. Third, we analyze how design choices such as state representation and reward shaping influence the behavior learned by the agents.

By treating *Plants vs. Zombies* as a case study rather than only as a game, this project aims to draw broader conclusions about reinforcement learning in resource-constrained sequential decision-making tasks. The same ideas studied here may be relevant to other problems where an agent must allocate limited resources, defend against uncertain future events, and make decisions with long-term consequences. Examples include automated defense planning, task scheduling, warehouse resource allocation, and other simulation-based strategy problems.

The rest of this report is organized as follows. Section ?? introduces the reinforcement learning concepts needed to understand the project. Section ?? describes the design of the *Plants vs. Zombies*-inspired environment, and the following sections present the reinforcement learning formulation (section ??), implementation (section ??), experimental setup (section ??), and results (section ??). The report then reviews related work (section ??) and discusses the findings in terms of learned behavior, comparison with baselines, state representation, reward design, training difficulty, and limitations (section ??). Finally, the report concludes with possible future improvements (section ??).

During this project, generative AI tools were used as supporting tools for writing and programming. For the report, AI assistance was used to polish paragraphs, improve wording, and help organize the presentation of ideas. The technical content, experimental setup, results, and conclusions were reviewed and decided by the project group. During implementation, AI tools were also used for code completion, debugging support, and explaining programming errors. The final code, experiments, analysis, and interpretation of results remain the responsibility of the project group.

II. BACKGROUND

This section introduces the reinforcement learning concepts needed to understand the rest of the report. We first describe reinforcement learning as a framework for sequential decision-making, then introduce the value-based and policy-based methods used in this project. We also discuss reward shaping, since the original task provides sparse feedback, and finally explain why games are commonly used as controlled testbeds for reinforcement learning research. These concepts provide the foundation for formulating the *Plants vs. Zombies*-inspired environment as a resource-constrained sequential decision-making problem.

A. Reinforcement Learning

Reinforcement learning (RL) is a framework for sequential decision-making in which an agent learns to act by interacting with an environment [?]. At each timestep t , the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$ according to a policy π , receives a scalar reward r_t , and transitions to a new state s_{t+1} . The agent's objective is to find a policy π^* that maximizes the expected cumulative discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad (1)$$

where $\gamma \in [0, 1)$ is a discount factor controlling the trade-off between immediate and future rewards. The action-value function $Q^\pi(s, a)$ represents the expected return when taking action a in state s and thereafter following policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid s_t = s, a_t = a]. \quad (2)$$

The optimal action-value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]. \quad (3)$$

The environment studied in this project has several properties that make classical tabular RL infeasible. The state representation is high-dimensional, the action space contains 316 discrete actions, and many rewards are delayed because the effect of

a plant-placement decision may only become visible later in the game. In addition, stochastic zombie spawning means that the agent must learn a policy that generalizes across different wave configurations rather than memorizing fixed patterns. These properties motivate the use of deep reinforcement learning methods that approximate value functions or policies with neural networks.

B. Deep Q-Networks and Double DQN

Deep Q-Networks (DQN) approximate the optimal action-value function $Q^*(s, a)$ using a neural network $Q(s, a; \theta)$ [?]. This makes Q-learning applicable to environments where the state space is too large for a tabular representation. DQN is commonly stabilized using experience replay and a target network. Experience replay stores previous transitions and samples them during training, while the target network provides more stable target values.

However, standard DQN can overestimate action values because the same network is used for both action selection and value estimation. Double DQN (DDQN) reduces this problem by decoupling these two operations [?]. The online network selects the greedy action, while the target network evaluates the value of that action:

$$y_t = r + \gamma, Q(\cdot, \arg \max_{a'} Q(s', a'; \theta); \theta^-). \quad (4)$$

DDQN is suitable for this project because the agent acts in a large but discrete action space. Each action corresponds either to waiting or to placing a specific plant at a specific grid location. Therefore, the DDQN agent can estimate one action value for each possible plant-placement decision.

C. Actor-Critic

Actor-Critic methods combine policy-based and value-based learning [?]. They maintain two components: an **actor** $\pi(a | s; \phi)$ that learns a policy, and a **critic** $V(s; \psi)$ that estimates state values. The critic evaluates the outcome of the actor's decisions and provides a learning signal for improving the policy. A common actor update is based on the policy gradient:

$$\nabla_{\phi} J(\phi) = \mathbb{E} * \pi [A(s_t, a_t), \nabla * \phi \log \pi(a_t | s_t; \phi)], \quad (5)$$

where the advantage can be estimated using the temporal-difference error:

$$A(s_t, a_t) = r_t + \gamma V(s_{t+1}; \psi) - V(s_t; \psi). \quad (6)$$

Actor-Critic methods are a standard approach for directly learning policies while still using value estimates to guide learning [?]. In this project, the Actor outputs a probability distribution over possible plant-placement actions, while the Critic estimates whether the current state is likely to lead to good long-term survival. This is useful in a stochastic environment because the policy can represent uncertainty over actions rather than always selecting a single maximum-value action.

D. Reward Shaping

Reward shaping adds intermediate rewards or penalties to help agents learn in environments where final rewards are sparse. This is important because an agent may otherwise receive little useful feedback until the end of an episode. However, reward shaping must be designed carefully, since poorly designed rewards can change the behavior the agent learns [?].

In this project, reward shaping is necessary because the final win/loss signal alone would provide limited feedback. The agent receives rewards for events such as killing zombies, clearing lanes, surviving waves, and preserving mowers. It also receives penalties for mower loss, dangerous zombie positions, and game over. These intermediate signals help the agent connect earlier plant-placement decisions to later survival outcomes. At the same time, the reward design can influence the learned strategy. For example, penalizing every plant placement may discourage useful stalling behavior, where temporary plants are used to delay zombies until stronger responses become available.

E. Games as Reinforcement Learning Testbeds

Games are widely used as reinforcement learning testbeds because they provide clear rules, measurable objectives, and repeatable experiments. The Arcade Learning Environment and OpenAI Gym are influential examples of platforms designed to evaluate RL agents in standardized game-like environments [?], [?].

Different game genres emphasize different RL challenges. Some games focus on fast reactions, while others require planning, resource allocation, or long-term strategy. A *Plants vs. Zombies*-inspired environment is useful for this project because it combines spatial placement, resource management, stochastic enemy waves, and delayed rewards. These properties make it a compact case study for evaluating how DDQN, Actor-Critic, and baseline agents handle resource-constrained sequential decision-making.

III. ENVIRONMENT DESIGN

The environment is designed as a simplified *Plants vs. Zombies*-inspired simulation for studying reinforcement learning in resource-constrained sequential decision-making. Rather than reproducing the full commercial game, the goal is to preserve the mechanics that are most relevant to RL: spatial placement, limited resources, stochastic events, delayed consequences, and increasing difficulty.

A. Game Mechanics

The game takes place on a 5×9 grid with five horizontal lanes and nine positions per lane. Zombies enter from the right side of the lawn and move left toward the house, while the agent defends by placing plants on grid cells. The episode ends in success if the agent survives all zombie waves, and in failure if a zombie reaches the house.

At each decision step, the agent observes the current game state and chooses either to wait or to place a specific plant at a specific grid location. This creates a sequential decision-making problem: the agent must decide not only which plant to use, but also when and where to place it. Since plants have different sun costs and cooldowns, the agent must balance immediate defense against long-term resource management.

Sun is the main resource in the environment. Some plants generate sun, while others provide damage, delay, or emergency defense. This creates a trade-off between investing in economy and responding to current threats. The lane structure also requires spatial reasoning, since threats appear in different lanes and must be handled locally. Lawn mowers provide a final emergency defense for each lane, but using one is treated as a negative event because it indicates that the agent failed to control that lane earlier.

These mechanics make the environment suitable as an RL case study. A plant-placement action may not show its full effect immediately, but it can influence survival several waves later. The agent must therefore learn policies that combine resource allocation, spatial placement, and delayed reward optimization.

B. Zombie Spawning Algorithm

To avoid fixed and memorized enemy patterns, the environment uses a budget-based zombie spawning algorithm. In order to achieve this, we use an algorithm that mimic how the original game behaves.

In our Environment, Each game session is divided into waves. At the start of each wave, the zombie director receives a budget based on the current wave index. As the wave index increases, the budget increases, allowing later waves to contain more zombies or stronger zombie types.

During a wave, the director repeatedly filters the zombie catalog for affordable zombies, selects one using weighted random selection, chooses one of the five lanes randomly, and spawns the zombie in that lane. The selected zombie's cost is then subtracted from the remaining wave budget. This process continues until the budget is exhausted.

For every 5 Wave, there's a Difficulty Spike in the form of Huge Wave where All Budgets are immediately used and Zombies are spawned in all 5 lanes simultaneously. This Huge Wave serve as a kind of Difficulty wall that the agent must pass everytime they reached a specific point in the episode.

This design gives the environment both structure and stochasticity. The increasing wave budget creates a difficulty curve, while random zombie and lane selection creates variation between episodes. As a result, the agent cannot rely on a fixed action sequence. It must instead learn a general defensive policy that can adapt to different enemy configurations.

C. Plant and Zombie Roster

The plant and zombie rosters are designed to create different types of decisions for the agent. The plant set contains seven plant types with different roles: resource generation, continuous damage, blocking, delayed damage, stronger damage, single-target elimination, and emergency lane clearing. For example, Sunflower supports long-term resource generation, Peashooter and Repeater provide sustained damage, Wallnut delays zombies, Potato Mine provides delayed elimination, Chomper removes a zombie after contact but has a recovery period, and Jalapeno clears an entire lane at high cost.

The zombie roster contains both standard and special enemies. Basic, Conehead, and Buckethead zombies mainly differ in durability. Additional special zombies introduce more varied threats: Pole-vaulter Zombie can jump over the first plant it encounters, Newspaper Zombie becomes faster after taking enough damage, and All-Star Zombie charges forward and deals heavy damage to the first plant it hits.

These rosters increase the strategic diversity of the environment without making the simulation unnecessarily complex. The agent must learn how plant roles match different enemy threats, when to save resources, and when to use emergency actions. In this sense, the environment represents a broader class of RL problems where an agent must allocate limited resources under uncertainty while responding to time-dependent risks.

D. Difficulty

The difficulty in the environment is defined by how much resources the opponent (Zombie Director) has access to and how aggressive they are. This is implemented by increasing the scaling of the budget. Budget scaling for one gameplay is determined by the formula

$$B(n) = \text{Int}(B(n-1) + n * \text{scale})$$

With n being the current number of waves and $B(0) = 1$. Int operator is used to make sure that the Scaling is rounded to integer since the lowest Zombie cost is 1.

Normal Difficulty uses the scale of 1, which means that the budget of each waves will be increased by n . Moderate Difficulty uses the scale of 1.5, and Hard Difficulty uses the scale of 2.

Aside from the budget scaling, the aggressiveness of the agent is also implemented by the spawn interval. In Normal and Moderate Difficulty the spawn interval for each zombies in a wave is 5 ticks, while in Hard Difficulty the spawn interval is reduced to 2 ticks. The Evaluation Environment is run on Normal Difficulty.

IV. RL FORMULATION

A. State space

The state is represented as a 126-dimensional vector encoding all information the agent needs to make informed placement decisions. Table ?? summarizes each component.

TABLE I
STATE SPACE COMPONENTS

Component	Description	Size
Plant Grid	Plant type at each grid cell	$5 \times 9 = 45$
Zombie Presence (per-grid)	Summed zombie HP at each cell	$5 \times 9 = 45$
Zombie Presence (per-lane)	Summed zombie HP per lane	5
Sun Amount	Current sun in possession	1
Plant Availability	Boolean availability per plant	7
Cooldowns	Per-plant cooldown in ticks	7
Mowers	Mower present per lane	5
Nearest Zombie	Normalized nearest zombie position per lane	5
Lane Plant HP	Summed plant HP per lane	5
Game Progress	current_wave/max_wave	1
Total		126

Compared to the 58-dimensional state used in prior work [?], this richer representation provides the agent with fine-grained spatial awareness (per-grid zombie HP), temporal awareness (cooldowns, game progress), and lane-level danger signals (nearest zombie position, mower status).

B. Action space

The action space is a flat discrete space of size:

$$|\mathcal{A}| = |\text{deck}| \times N_{\text{lanes}} \times \text{lane_length} + 1 = 7 \times 5 \times 9 + 1 = 316 \quad (7)$$

Action 0 is a *wait* action, allowing the agent to do nothing for the current tick. This is essential for resource management: the agent must learn when to save sun rather than spending it immediately. Actions 1–315 each correspond to placing a specific plant type at a specific lane and column position.

C. Reward and penalty design

The reward function combines two categories of signals: bulk rewards triggered by discrete events, and ongoing rewards accumulated per tick.

Zombie kill rewards are scaled by the wave budget rather than fixed values, so that eliminating stronger late-game zombies yields higher reward. The mower penalty and survival reward together encourage the agent to maintain active defenses rather than relying on mowers as a fallback. The danger score provides a continuous shaping signal that penalizes the agent for allowing zombies to advance deep into the lawn, mitigating the sparse reward problem inherent in a game where the terminal penalty only occurs at lane breach.

Besides the rewards defined on table ??, we also tried several rewards that didn't perform as well and therefore unused on the final agent. Those reward functions are:

- Penalty for each Sun Used

TABLE II
REWARD AND PENALTY COMPONENTS

Type	Signal	Kind	Description
Bulk	Zombie Kill	Positive	Scaled by zombie wave budget
Bulk	Lane Clear Bonus	Positive	All zombies cleared in a lane
Bulk	Game Over	Negative	Zombie reaches the house
Bulk	Mower Triggered	Negative	Moderate penalty per mower used
Ongoing	Mower Survival	Positive	Reward per tick each mower survives
Ongoing	Wave Survival	Positive	Reward per wave cleared
Ongoing	Danger Score	Negative	Penalty scaled by nearest zombie proximity

- Penalty for each Planted Plant
- Penalty for each Plant Death

The hypothesis for why these rewards are detrimental to the agent is because in the environment Investments are more rewarding than saving. Agents that managed to plant enough sun producing plants in the beginning and used them to plant a lot of defensive plants when they can, perform much better than the one that saves resources.

D. Invalid Action Masking

To address the large action space and the sparsity of valid actions, we implement invalid action masking for all learning agents. At each timestep, the environment provides a boolean mask indicating which of the 316 actions are currently legal. The "wait" action is always valid, while planting actions are only valid if the agent has sufficient sun, the selected plant is not on cooldown, and the target grid cell is empty.

For the DDQN agent, the mask is applied during the greedy action selection phase. The Q-values for all invalid actions are set to a large negative number before taking the argmax, ensuring that an invalid action is never selected as the best action. During the exploration phase, the agent randomly selects only from the pool of valid actions. When computing the target Q-values for the Bellman update, invalid actions in the next state are similarly masked out to prevent the agent from bootstrapping values from impossible future states.

For the Actor-Critic agent, the mask is applied directly to the policy network's output logits. The logits corresponding to invalid actions are masked with a large negative value (-10^9) before the softmax activation is applied. This guarantees that the probability of selecting any invalid action is exactly zero, allowing the policy gradient to focus entirely on optimizing the distribution over valid actions.

V. IMPLEMENTATION

This section describes the agents implemented and compared in the project. The purpose of this section is to explain how each agent produces actions in the environment. The evaluation protocol and performance metrics are described separately in Section ??.

All agents interact with the same environment interface. At each decision step, an agent receives the current state representation and returns one action from the discrete action space. The action is either to wait or to place a specific plant at a specific location on the 5×9 grid. With seven plant types, five lanes, nine positions, and one wait action, the action space contains $|A| = 7 \times 5 \times 9 + 1 = 316$ possible actions.

A. DDQN Agents

We use Double Deep Q-Networks (DDQN) as value-based reinforcement learning agents. The DDQN agent receives the 126-dimensional state vector as input and outputs one Q-value for each of the 316 possible actions. The selected action is the one with the highest predicted Q-value, except during exploration, where an ϵ -greedy strategy is used.

The DDQN implementation uses an online network and a target network. The online network selects actions, while the target network is used to compute target Q-values, following the DDQN method proposed by van Hasselt et al. [?]. Transitions of the form $[(s_t, a_t, r_t, s_{t+1}, done)]$ are stored in replay memory and sampled in mini-batches during training.

Two DDQN-based agents are included in the comparison. The first is *DDQN Legacy*, which refers to the DDQN model from the previous *Plants vs. Zombies* reinforcement learning implementation. This model is used as a reference point and only supports the smaller original plant set. The second is *DDQN New*, which adapts the legacy DDQN approach to the extended environment and larger action space. This agent is trained further in the new environment for 10^5 iterations.

B. Actor-Critic Agent

The Actor-Critic agent is implemented as a policy-based reinforcement learning agent with a separate value estimator. The Actor receives the 126-dimensional state representation and outputs a probability distribution over the 316 possible actions. An action is then selected from this distribution. The Critic receives the same state and estimates its expected future return.

After each environment step, the Critic computes a temporal-difference error, which is used to update both the Critic’s value estimate and the Actor’s policy. Actions that lead to better-than-expected outcomes become more likely, while actions that lead to worse outcomes become less likely. In the current implementation, the Actor-Critic agent is trained for 10^5 iterations on a slightly harder version of the environment in order to encourage more robust behavior under stronger wave pressure.

C. Non-Learning Baselines

Two simple non-learning agents are implemented as lower-bound baselines. The *DummyDoNothing* agent always selects the wait action and never places plants. It represents the weakest possible strategy and is used to confirm that survival requires active decision-making.

The *DummyRandom* agent selects randomly from available actions. Unlike the do-nothing agent, it interacts with the environment by placing plants, but it does not use the state strategically and does not learn from rewards. This baseline shows how much performance can be achieved through unguided action selection.

D. Heuristic Agent

A rule-based heuristic agent is implemented as a stronger non-learning comparison. Unlike the random and do-nothing baselines, the heuristic agent uses information from the state representation to select actions according to manually designed rules. It does not learn from experience, but it represents a simple expert-designed strategy.

The heuristic prioritizes economy, lane defense, and emergency responses. In the early game, it attempts to plant Sunflowers frequently and maintain at least a minimum number of Sunflowers on the lawn. When zombies become dangerous, it prioritizes instant-response plants such as Jalapeno or Chomper. It also places medium-cost attackers such as Peashooters in lanes with concentrated zombie pressure and uses stronger attackers such as Repeaters more often in the mid and late game.

This heuristic agent provides a more meaningful comparison than the trivial baselines. If a reinforcement learning agent performs better than the heuristic, this suggests that learning has produced behavior beyond simple manually designed rules. If the heuristic performs similarly or better, it indicates that the learned policy may still require better training, reward design, action masking, or state representation.

VI. EXPERIMENTAL SETUP

A. Training setup

The DDQN and Actor-Critic agents were trained in the custom *Plants vs. Zombies*-inspired environment described in the previous sections. Each training episode corresponds to one complete game session. An episode starts from the initial lawn state and ends when either the agent survives all zombie waves or a zombie reaches the house.

Both learning agents used the same environment configuration. This includes the same plant roster, zombie roster, zombie spawning algorithm, state representation, action space, and reward function. The state input is the 126-dimensional feature vector described in Section ??, and the action space contains 316 discrete actions: one wait action and 315 plant-placement actions over seven plant types and the 5×9 grid.

During training, zombie waves were generated by the budget-based spawning algorithm. As a result, the agents were not trained on a fixed sequence of zombies. Instead, zombie types and lane positions could vary between episodes. This stochastic setup was used to encourage the agents to learn general defensive strategies rather than memorize a specific wave pattern.

The same reward function was used for both learning agents. It includes event-based rewards, such as zombie kills, lane clearing, wave survival, mower penalties, and game-over penalties, as well as ongoing rewards and penalties, such as mower survival and danger-based penalties. These intermediate rewards were used because the final win/loss signal alone would be too sparse for efficient learning.

During training, we recorded survival-related indicators such as ticks survived and waves reached. These values were used to monitor whether the agents improved over time during the training process.

B. Evaluation setup

After training, the agents were evaluated separately from the training process. No additional learning was performed during evaluation. The goal was to measure how well the final trained policies performed across multiple independent game sessions.

Each agent was evaluated over 100 game sessions. Because the zombie spawning algorithm is stochastic, each session could contain different zombie types and lane placements. Evaluating over multiple sessions gives a more reliable estimate of performance than using a single run, since one game may be affected by unusually easy or difficult zombie spawns.

The evaluated agents were DDQN, Actor-Critic, *DummyRandom*, and *DummyDoNothing*. The two learning agents used their trained policies to select actions. The *DummyRandom* agent selected actions randomly from the action space, while the *DummyDoNothing* agent always selected the wait action. These baselines provide reference points for determining whether the learning agents developed useful behavior.

All agents were evaluated under the same environment rules, plant and zombie types, grid size, reward function, and wave-generation procedure. The results were then summarized using survival and score-based metrics over the 100 evaluation sessions.

C. Metrics

The agents were compared using several metrics. *Ticks survived* measures how long the agent keeps the game running before losing or completing the episode. By adjusting cooldown, speed, and other time-based parameters in the environment, we tried to mimic the actual Plant vs Zombies game as close as possible, where in the simulated environment 1 Ticks roughly equals to 1 Seconds in the real game.

Second evaluation metric is *Waves reached* which measures how far the agent progresses through the wave sequence, which is important because later waves are more difficult and there's difficulty wall in the form of Huge Wave.

Episode score or total accumulated reward summarizes the rewards and penalties collected during a session to see how well our model performs during the training. This is also used to evaluate whether the reward and penalties given to the agent is good enough to approximate the reward of the environment and the goal of the agent to survive longer.

VII. RESULTS

This section presents the experimental results used to answer the research questions. We focus on survival-based metrics rather than raw reward score, since the reward value depends on the chosen shaping terms and does not have a direct interpretation by itself. The main metrics are ticks survived and waves reached, which measure how long the agent survives and how far it progresses through the increasing difficulty curve.

A. Training performance

Figure ?? shows the training performance of the best-performing Actor-Critic agent. The Reward Score increase proportionally and have similar shape to the Evaluation Metrics (Ticks & Waves) which means that the reward function was able to approximate the correct interim reward for the environment.

During the early phase of training, from episode 0 to approximately episode 20,000, the agent improves rapidly in both ticks survived and waves reached. This suggests that the agent quickly learns basic survival behavior, such as early resource generation and simple lane defense.

Between approximately episode 20,000 and episode 68,000, the learning curve enters a more stable plateau. Around episode 70,000, the agent experiences a temporary performance collapse, with survival duration dropping below 200 ticks and waves reached dropping to around 5. However, the agent recovers afterward and eventually reaches its best performance near episode 100,000. This pattern suggests that training is not fully stable, but that the agent can recover from temporary policy degradation and continue improving.

B. Effect of Reward Shaping

Figure ?? compares agents trained with different reward designs over 100 evaluation sessions. The agent trained with additional interim rewards performs better than the agent trained only with sparse rewards based on survival and zombie-kill events. This indicates that reward shaping helps the agent learn in an environment where the final outcome is delayed and many useful actions do not produce immediate success.

The result supports the idea that intermediate rewards reduce the credit-assignment problem. By rewarding progress during the episode, such as surviving waves, preserving mowers, and responding to local danger, the agent receives more frequent feedback about which actions contribute to long-term survival. This makes the learned policy more robust than one trained only from sparse event-based feedback.

C. Effect of State Representation

Figure ?? shows the effect of state representation on performance. The agent trained with the more detailed state representation performs slightly better than the agent trained with the summarized representation. The detailed representation includes more granular information, such as zombie health per grid cell, nearest zombie positions, plant cooldowns, mower status, and game progress.

This suggests that richer state information helps the agent make better decisions in a spatial and time-dependent environment. In particular, detailed zombie-position information is useful because the agent must decide not only which plant to place, but also where and when to place it. However, the improvement is moderate, indicating that increasing state detail alone is not sufficient; the agent must also be able to learn effectively from the additional information.

figures/AC-Training_combined.png

Fig. 1. Training Performance for Best Performing Agent

figures/Reward Function.jpg

Fig. 2. Performance Difference between Agent trained with Simple Reward vs Added Interim Rewards

figures/State Representation.jpg

Fig. 3. Performance Difference between Agent trained with Simple State Representation vs A More Detailed State Representation

figures/Difficulty.jpg

Fig. 4. Performance Comparison between Agent trained using Normal Difficulty (1), Moderate Difficulty (2), and Hard Difficulty (3)

D. Effect of Training Difficulty

Figure ?? compares agents trained under different difficulty settings. Agents trained on harder difficulty show smaller performance variation, suggesting that exposure to more challenging waves can encourage more consistent strategies. This supports the idea that training difficulty influences not only average performance, but also the stability of the learned policy.

However, the most difficult setting also reduces the agent’s maximum performance. This suggests a trade-off: harder training can improve consistency, but if the environment becomes too difficult, the agent may not experience enough successful outcomes to learn high-performing behavior. A moderate difficulty level therefore appears more useful than either an easy setting or an excessively hard setting.

E. Comparison Across Agents

Figure ?? compares the best-performing agents against baseline and benchmark agents over 100 evaluation sessions. The compared agents include *DummyDoNothing*, *DummyRandom*, the previous-work DDQN benchmark, the new DDQN agent, the heuristic agent, and the Actor-Critic agent.

The Actor-Critic agent achieves the strongest overall performance among the evaluated agents. It consistently outperforms the trivial baselines and also performs better than the heuristic and previous-work DDQN benchmark in the plotted evaluation. This suggests that the learned Actor-Critic policy is able to discover useful resource-management and spatial decision-making strategies in the stochastic lane-defense environment.

The comparison with the heuristic agent is especially important. Beating the random and do-nothing baselines only shows that the agent learned behavior better than unguided or inactive action selection. Performing better than a rule-based heuristic provides stronger evidence that reinforcement learning can discover strategies beyond simple manually designed rules in this environment.

Compared to the human baseline (averaged across 10 manual trials within the environment), the Actor-Critic agent demonstrated competitive performance, with the majority of failures occurring near or above the benchmark. However, it should be noted that human performance was inherently constrained by slower reaction times and suboptimal control interfaces. Consequently, this baseline should be interpreted with caution.

Lastly, Episode termination are highly clustered with noticeable intervals in between. This behavior aligns with expectations, as most agents are defeated at the ‘Huge Waves’ which act as severe difficulty barriers. Even agents that do not fail at these exact points typically have their defenses entirely decimated, particularly during later stages. Furthermore, this aspect also provides a probable explanation for the legacy model poor performance. These severe difficulty spikes didn’t exist on the legacy environment. Which when combined with zombies with special abilities that the agents never encounter, severely disadvantages the legacy agent.

VIII. RELATED WORK

This section positions the project within previous work on reinforcement learning for sequential decision-making. We first discuss deep reinforcement learning in games as a general research direction, then narrow the focus to tower-defense and *Plants vs. Zombies*-like environments. Finally, we discuss reward shaping and invalid action masking, which are important design considerations for applying RL to large discrete action spaces with sparse rewards.

A. Deep Reinforcement Learning in Games

Deep reinforcement learning has been widely studied in game environments because games provide clear objectives, controlled rules, and repeatable evaluation. More importantly, many games represent general sequential decision-making problems: an agent must observe the current state, choose an action, and optimize long-term outcomes under uncertainty. This makes games useful not only as entertainment domains, but also as testbeds for studying planning, exploration, generalization, and delayed rewards.

A major milestone in this area was the development of Deep Q-Networks (DQN). Mnih et al. showed that a neural network could approximate action values and learn successful policies directly from high-dimensional Atari game inputs [?]. This demonstrated that deep reinforcement learning could be applied to complex decision problems where handcrafted state-action tables are infeasible. The Arcade Learning Environment further supported this direction by providing a standardized benchmark for evaluating general agents on Atari games [?]. OpenAI Gym later introduced a broader interface for comparing RL algorithms across many environments [?].

Our project follows this general line of work, but focuses on a different type of decision problem. Many Atari tasks emphasize fast reactions and direct control, while our environment emphasizes resource allocation, spatial placement, stochastic threats, and delayed consequences. In this sense, the *Plants vs. Zombies*-inspired environment is used as a compact case study for a broader class of RL problems where an agent must allocate limited resources over time.

figures/Final Comparison.jpg

Fig. 5. Performance comparison between best performing agents against baselines and benchmarks

B. Reinforcement Learning for Resource-Constrained and Tower-Defense Environments

Tower-defense games are closely related to resource-constrained planning problems. In such environments, an agent must decide where and when to deploy defensive units, how to allocate limited resources, and how to respond to threats that evolve over time. These properties make tower-defense games relevant to broader RL applications such as scheduling, automated defense planning, inventory control, and other domains where actions have long-term consequences.

The most directly related work is a previous project that applied reinforcement learning to a simplified *Plants vs. Zombies* environment [?]. That work formulated the game as an RL task with discrete plant-placement actions, zombie waves, and resource constraints. Its environment used a smaller setup with three zombie types, four plant types, randomized spawning, and an action space of 181 actions. The project reported that DDQN performed strongly in high-difficulty settings and that valid-action masking improved performance by preventing the agent from selecting impossible moves.

More recent work has studied reinforcement learning for high-level strategic control in tower-defense games. Bergdahl et al. investigated RL in tower-defense settings and tested their approach on *Plants vs. Zombies* [?]. Their work combined reinforcement learning with scripted heuristic behavior and showed that hybrid approaches can outperform purely heuristic agents. They also emphasized that training general agents for puzzle-like tower-defense games remains difficult. This is relevant to our project because our environment also combines spatial placement, resource management, stochastic enemy waves, and long-term planning.

Compared with these works, our project uses *Plants vs. Zombies* not only as a game environment, but as a representative example of a broader RL problem: resource-constrained sequential decision-making under uncertainty. The goal is therefore not simply to make an agent play the game, but to study how RL agents behave when they must balance economy, defense, stochastic events, and delayed rewards.

C. Reward Shaping and Invalid Action Masking

Reward design is a central issue when applying reinforcement learning to environments with sparse or delayed feedback. In many sequential tasks, the final outcome may be easy to define, but it may not provide enough information for efficient learning. Reward shaping addresses this by adding intermediate rewards or penalties that help guide the agent during training. Ng et al. showed that reward transformations must be designed carefully, since poorly designed shaping can change the intended optimal behavior [?].

This issue is directly relevant to our environment. The final outcome is whether the agent survives all waves or loses when a zombie reaches the house, but this signal alone gives limited feedback about which earlier plant-placement decisions were useful. Therefore, our environment includes shaped rewards for zombie kills, lane clearing, wave survival, and mower preservation, as well as penalties for mower loss, dangerous zombie positions, and game over. At the same time, our experiments suggest that reward shaping can have unintended effects. For example, penalizing every plant placement may discourage useful stalling behavior, where temporary plants delay zombies until stronger responses become available.

Invalid action masking is another important technique for environments with large discrete action spaces. In many RL problems, not all actions are legal in every state. Huang and Onta n'on studied invalid action masking and showed that it becomes increasingly important as the number of invalid actions grows [?]. This applies directly to our environment because many of the 316 actions may be invalid at a given time: the agent may lack enough sun, the selected plant may be on cooldown, or the target cell may already be occupied. Previous *Plants vs. Zombies* RL work also reported that valid-action masking improved performance, making it a natural extension for similar project.

D. Summary of Differences from Previous Work

The related work shows that reinforcement learning has been successfully applied to games as controlled decision-making environments, and that tower-defense games provide a useful setting for studying resource allocation, spatial planning, and delayed rewards. Our project builds on this direction but differs from previous work in several ways.

First, our environment increases the complexity of the earlier simplified *Plants vs. Zombies* setup. We use seven plant types instead of four and include additional plant roles such as stronger continuous damage, one-hit elimination with recovery time, and expensive lane-wide emergency clearing. We also add special zombies with different mechanics, including jumping, speed-up behavior, charging attacks, and higher durability. These changes create a more diverse set of tactical situations for the agent.

In addition to that, we also completely changes the behavior of the opponent (Zombie Director). While the previous work only use uniform random for Zombie and lane selection, the opponent in our environment uses an algorithm that much more closer to how the original game behaves, more specifically by using Wave Budgeting which are explained in detail on section ?? . These changes forces the agent to be more strategic and more importantly introduce the aspect of increasing difficulty.

Second, our RL formulation uses a richer state representation. The state includes plant grid information, zombie health per grid and per lane, sun amount, plant availability, cooldowns, mower status, nearest zombie positions, lane plant health, and



figures/game_session.png

Fig. 6. Average gameplay of the best performing agent

game progress. This expands the state representation from 58 to 126 dimensions. The richer representation gives the agent more information about the current situation, but also increases learning complexity.

Third, our action space is larger. With seven plant types, five lanes, nine positions, and one wait action, the agent chooses from 316 discrete actions. This makes exploration more difficult and strengthens the motivation for future improvements such as valid-action masking or hierarchical action selection.

Finally, our project compares DDQN and Actor-Critic agents against simple non-learning baselines in a stochastic wave-based environment. Rather than focusing only on whether an RL agent can play *Plants vs. Zombies*, we use the game as a case study to analyze how state representation, reward design, action-space size, and stochastic difficulty affect learning in resource-constrained sequential decision-making tasks. This broader framing connects the project to other possible applications where an agent must allocate limited resources under uncertainty and optimize long-term outcomes.

IX. DISCUSSION

A. Agent Behavior

By observing the gameplay of the best performing agent over several gameplay iterations. There's several strategies that the agent often takes. As shown in Figure ??, In the initial stage of an episode, the agent consistently prioritizes resource generation by planting Sunflowers, deploying only the minimum required defenses. This demonstrates a learned exploitation of early, low-threat environments to maximize cumulative future returns.

As episode length increases and difficulty scales, the action distribution shifts strictly toward defense creation and structural replacement. Actions associated with delayed utility, such as planting Sunflowers or Potato Mines, are systematically lowered in advanced stages in favor of immediate board stabilization.

The agent also developed specific tactical responses to immediate threats. When zombies get dangerously close to the main defenses, the agent actively places available lower-cost plants to stall their advance until instant-kill options, like the Jalapeno or Chomper, are ready to deploy. Additionally, the agent consistently places plants on the rightmost edge of the grid. This might be a direct result of the introduction of All-Star and Pole Vaulter zombie as it's a reliable countermeasure against both of their gimmicks.

Lastly, the agent demonstrates a strong evaluation of long-term tradeoffs by occasionally sacrificing a Lane Mower. If early zombie spawns are concentrated in a single lane, the agent will often let the mower trigger rather than waste resources on suboptimal early defense. This behavior indicates that the agent has learned that the long-term reward of early resource scaling outweighs the immediate penalty of losing the mower.

B. Reinforcement Learning as a Solution for Resource-Constrained Sequential Decision-Making

The results suggest that reinforcement learning can learn useful strategies in a resource-constrained sequential decision-making environment. The best-performing Actor-Critic agent did not only react to immediate threats, but also learned behavior connected to long-term planning. This indicates that the agent learned the value of investing in future resources while still maintaining enough short-term protection.

The observed stalling behavior also supports this interpretation. This type of behavior is important because it shows that the agent can learn strategies where the value of an action is delayed. Similarly, the agent sometimes appeared willing to sacrifice a mower in exchange for preserving resources and building a stronger long-term economy. These patterns suggest that RL can capture trade-offs between immediate penalties and future return.

C. Comparison with Baselines and Heuristic Agents

The comparison with baseline agents helps show whether the learned policies are meaningful. The *DummyDoNothing* agent represents the lowest reference point, since it never responds to the environment. The *DummyRandom* agent provides a stronger but still unguided comparison, since it selects actions without using the state strategically. The learning agents outperforming these baselines indicates that they learned a useful relationship between state information and defensive actions.

The heuristic agent provides a more meaningful comparison because it uses manually designed rules, such as prioritizing Sunflowers early, placing attackers in threatened lanes, and using emergency plants when zombies become dangerous. The Actor-Critic agent performing better than the heuristic suggests that reinforcement learning can discover strategies that go beyond simple hand-coded rules in this environment. However, the heuristic result should not be interpreted as a final comparison against all rule-based approaches, since a more advanced heuristic or hybrid RL-heuristic method could perform differently.

D. Effect of State Representation and Reward Frequency

The experiments show that design choices in the RL formulation strongly affect performance. The agent trained with the richer state representation performed better than the agent trained with a summarized state. This is reasonable because the task requires spatial and temporal awareness. Information such as zombie health per grid cell, nearest zombie position, mower status, cooldowns, and game progress helps the agent distinguish between situations that may require different actions.

Reward frequency also had a clear effect. The agent trained with interim rewards performed better than the agent trained only with sparse rewards. This suggests that reward shaping helps reduce the credit-assignment problem. Many useful actions, such as planting Sunflowers early or delaying zombies with temporary plants, may only improve survival later in the episode. Intermediate rewards give the agent more frequent feedback about whether its actions contribute to long-term success.

At the same time, reward shaping must be designed carefully. A reward penalty for every plant placement may encourage resource efficiency, but it can also discourage useful stalling behavior. This shows that reward design does not only affect learning speed; it also shapes the strategy the agent learns.

E. Effect of Difficulty and Generalization

Training difficulty also influenced agent behavior. Agents trained on harder settings showed smaller performance variation, suggesting that exposure to stronger waves can encourage more consistent strategies. This indicates that challenging environments may help agents learn more robust behavior.

However, the most difficult setting reduced the agent’s maximum performance. This suggests that there is a trade-off between difficulty and learnability. If the environment is too easy, the agent may learn strategies that do not generalize to harder cases. If it is too difficult, the agent may fail too often and receive too little useful feedback. A gradual curriculum, where the environment becomes harder over time, may therefore be a better training strategy.

Generalization remains important because zombie spawning is stochastic. The agent cannot simply memorize one fixed sequence of actions; it must respond to different zombie types and lane configurations. The evaluation over multiple game sessions gives evidence that the learned policy can generalize across different sampled waves, although more repeated runs would be needed for stronger conclusions.

F. Limitations

This project has several limitations. First, the environment is a simplified simulation inspired by *Plants vs. Zombies*. It preserves important RL-relevant mechanics such as spatial placement, resource management, stochastic waves, and delayed rewards, but it does not reproduce the full original game. There’s also an action that is possible in the original game, Shovel, which is not implemented in the simplified simulation.

Second, the agent training is limited by device and time. As shown on figure ??, the model is still on increasing trend in the final episode (Episode 100,000), which means that it might achieve a better performance if training continues. But since plateau often happens and there’s no real way to know if the agent can perform better with further episode, we set a hard stop to the training.

Third, the evaluation is limited in scale. Testing over 100 game sessions provides useful evidence, but stronger conclusions would require multiple training seeds, repeated experiments, and statistical testing. This is especially important because deep reinforcement learning can be sensitive to initialization, exploration, and hyperparameter choices.

X. CONCLUSION AND FUTURE WORK

This project investigated whether reinforcement learning agents can learn effective resource-management and spatial decision-making strategies in a stochastic lane-defense environment. A simplified *Plants vs. Zombies*-inspired simulator was used as a case study because it contains several challenges common to sequential decision-making problems: limited resources, delayed rewards, stochastic events, spatial placement, and increasing difficulty.

The results show that reinforcement learning can learn meaningful defensive behavior in this environment. Among the evaluated agents, the Actor-Critic agent achieved the strongest overall performance compared with simple baselines, a heuristic agent, and DDQN-based benchmark agents. The observed behavior of the best agent suggests that it learned more than immediate reaction. It invested in Sunflowers early, shifted toward defense in later waves, used temporary plants to stall threats, and sometimes accepted short-term penalties for longer-term benefit. These behaviors indicate that the agent learned strategies related to long-term return, resource allocation, and adaptive defense.

The experiments also show that implementation and environment design choices strongly affect learning. A richer state representation improved performance by giving the agent more detailed information about the tactical situation. Interim rewards improved learning compared with sparse rewards, showing the importance of reward shaping in environments with delayed feedback. Training difficulty also affected policy consistency: harder training conditions encouraged more stable behavior, but excessive difficulty reduced overall performance. These findings suggest that reinforcement learning performance depends not only on the selected algorithm, but also on how the environment, state representation, rewards, and difficulty are designed.

There are several directions for future work. First, the model architecture could be improved. As shown on figure ??, A Collapse happen in the middle of the training. This can potentially be solved by using Trust Region from Trust Region Policy Optimization (TRPO) or Gradient Clipping from Proximal Policy Optimization (PPO). Both Architecture potentially will make the agent more robust and avoid bad-step to cause collapse such as portrayed.

Second, as we can see with the performance of the DDQN Legacy in figure ??, different environment can cause massive performance difference because of the opponent behaviour. In this case, changing the opponent (zombie director) algorithm and adding special zombies that the agent never saw in training managed to collapse the performance of the legacy agent completely. Future works could try to solve this generalization problem, as the roster that we currently use is less than 10% of the actual roster of the actual game environment and the agent likely won’t perform as well if we introduce another zombie type to the environment. The current agent is only generalized to the current environment, not to all environment.

Third, future work could explore curriculum learning. Instead of training agents on the same difficulty settings for the entire training session, we can start the training from easy difficulty and slowly increase the difficulty with each episodes as the agent start to learn new strategy and behaviour. Corresponding to our results on ??, this would allow the agent to get the benefit of

a more consistent strategy by training on higher difficulty while handling the trade-off of lower performance by allowing them to learn a concrete strategy before being thrust into a hard environment.

Fourth, the state and action representation could be improved. The current state is represented as a flat feature vector, but the environment has a clear spatial structure which might be lost in the current implementation. Combining the agent with Spatial-aware model such as Convolutional Neural Network (CNN) or Transformers with Positional Encoding might be beneficial.

Moreover, the currently large amount of action space might be detrimental to the agent, and by restructuring it into a spatially-correlated action map, the network can naturally exploit the geometric logic of the board. In this environment, besides some special cases (like Zombie being on the last columns) deploying a defensive unit at (1,1) yields a strategically analogous outcome to (1,2) since it's close and within the same lane. However, the action would significantly differs to deploying in different lane (2,1) or at an exposed frontline (1,9). An architecture that preserve the 2D topology would allow the model to share learned Q-values across adjacent, semantically similar regions, potentially reducing the exploration burden.

Fifth, more systematic reward ablation experiments should be conducted. Although interim rewards improved performance, the results also suggest that some reward components can change the learned strategy in unexpected ways. Testing each reward component separately would make it clearer which rewards are most important and which may introduce unwanted behavior.

Finally, the ideas studied in this project could be transferred to other resource-constrained sequential decision-making problems. The same structure appears in domains such as automated defense planning, warehouse scheduling, traffic control, inventory management, and simulation-based resource allocation. In these problems, an agent must make decisions under uncertainty, allocate limited resources, and optimize long-term outcomes. The *Plants vs. Zombies*-inspired environment therefore serves not only as a game case study, but also as a compact example of a broader class of problems where reinforcement learning may be useful.